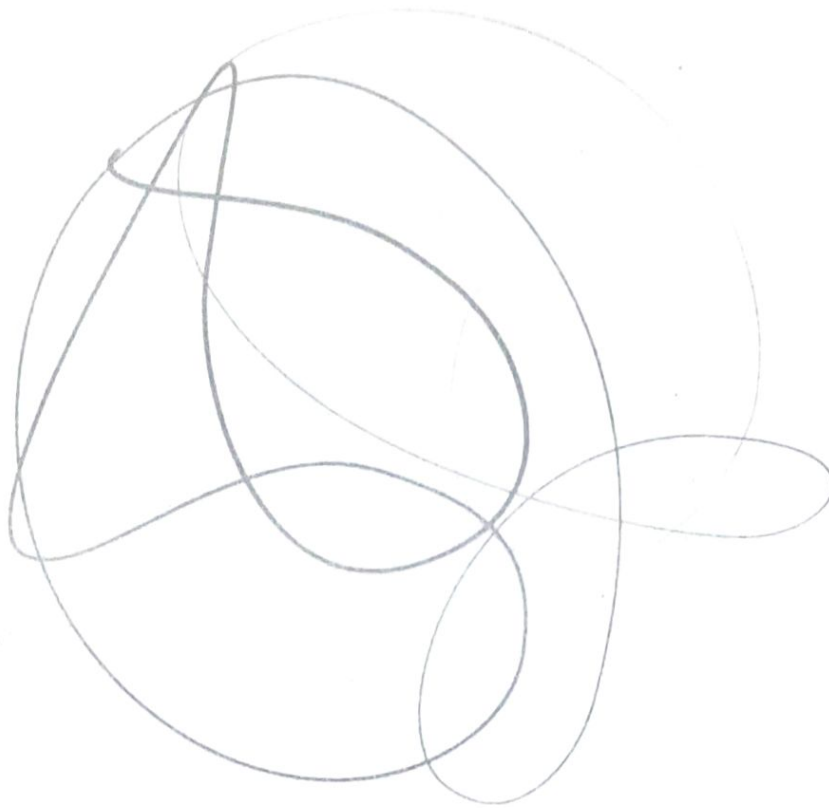




**Online Shopping
Application—Functional
API Build**

Version Dated: 05.2026

Table of Contents	3
Problem Statement	3
Functional Requirements	3
Database Design	3
Users Table	3
Categories Table	3
Products Table	4
CartItems Table	4
Orders Table	4
OrderDetails Table	4
Suggested Application Layers	5
Suggested Project Structure	5
Sample API Endpoints	6
Validations	7
Technology Stack	7
Non-Functional Requirements	7
Guidelines	7
Milestones	7

Problem Statement

ABC is expanding its successful physical store to an online platform. This case study focuses on **designing and implementing the User Module of an Online Shopping Application** using Python FastAPI.

The objective is to build a functional backend API that allows users to:

- register and log in,
- browse and search products,
- manage a cart,
- and place orders.

Use the following concepts to build the solution to the given case study:

- FastAPI project setup
- Request-response lifecycle understanding
- Repo and folder structure
- SQLAlchemy models
- Pydantic schemas
- CRUD API implementation
- Controller / Service / Repository layering
- ER diagram and entity mapping
- Relationships and joins
- Swagger / OpenAPI testing
- Docker + Docker Compose with PostgreSQL
- Basic exception handling and validations

Functional Requirements

Database Design

Users Table

- UserID (PK – Auto Generated)
- Name
- Email (Unique)
- Password
- Mobile

Categories Table

- CategoryID (PK – Auto Generated)
- CategoryName (Unique)

Products Table

- ProductID (PK – Auto Generated)

- ProductName (Unique)
- Description
- CategoryID (FK)
- Price
- AvailableQuantity
- ProductUrl

CartItems Table

- CartItemID (PK – Auto Generated)
- UserID (FK)
- ProductID (FK)
- Quantity

Orders Table

- OrderID (PK – Auto Generated)
- UserID (FK)
- OrderDate
- PaymentMethod
- TotalAmount

OrderDetails Table

- OrderDetailID (PK – Auto Generated)
- OrderID (FK)
- ProductID (FK)
- Quantity
- Price

Suggested Application Layers

The case study expects layered architecture with API, business logic, and data access layers. For FastAPI, use this structure:

- API Layer
- FastAPI routers/endpoints
- Service Layer
- Business logic and validations
- Repository Layer
- Database interaction methods
- Data Layer
- SQLAlchemy models + Postgres
- Schema Layer

- Pydantic request/response models

Suggested Project Structure

```
app/  main.py
      db/
      |  ├── session.py
      |  ├── base.py
      models/
      |  ├── user.py
      |  ├── category.py
      |  ├── product.py
      |  ├── cart.py
      |  ├── order.py
      schemas/
      |  ├── user_schema.py
      |  ├── product_schema.py
      |  ├── cart_schema.py
      |  ├── order_schema.py
      repositories/
      |  ├── user_repository.py
      |  ├── product_repository.py
      |  ├── cart_repository.py
      |  ├── order_repository.py
      services/
      |  ├── user_service.py
      |  ├── product_service.py
      |  ├── cart_service.py
      |  ├── order_service.py
      routers/
      |  ├── user_router.py
      |  ├── product_router.py
      |  ├── cart_router.py
      |  ├── order_router.py
      utils/
      |  ├── exceptions.py
      |  ├── helpers.py
      tests/
```

Sample API Endpoints

User APIs

- `POST /api/users/register` – Register a new user
- `POST /api/users/login` – Login using email and password

Product APIs

- `GET /api/products` – List all products
- `GET /api/products/{id}` – Get product details
- `GET /api/products/search?name=&category=` – Search products

Category APIs

- `GET /api/categories` – List all categories

Cart APIs

- `GET /api/cart/{user\_id}` – View user cart
- `POST /api/cart/add` – Add item to cart
- `PUT /api/cart/update/{cart\_item\_id}` – Update cart quantity
- `DELETE /api/cart/remove/{cart\_item\_id}` – Remove item from cart
- `GET /api/cart/{user\_id}/summary` – View cart summary

Order APIs

- `POST /api/orders/checkout` – Place an order
- `GET /api/orders/{user\_id}` – View order history
- `GET /api/orders/details/{order\_id}` – View order details

Validations

Appropriate validations should be applied during registration, login, cart operations, and order placement. Following are the recommended validations:

User Validations

- Name cannot be empty
- Email must be valid format
- Email must be unique
- Password must meet minimum length
- Mobile must be numeric and valid length

Product / Cart Validations

- Product must exist before adding to cart
- Quantity must be greater than 0
- Quantity cannot exceed available stock
- Cart item must exist before update/delete

Order Validations

- User must have at least one cart item before checkout
- Ordered quantity must not exceed available quantity
- Payment method must be valid
- Order total must be calculated correctly

Technology Stack

Framework	Python Fast API
Programming Language	Python 3.12
IDE	Visual Studio Code
Data Source	SQLite and PostgreSQL

Non-Functional Requirements

The solution calls for low latency, high throughput, scalability, and maintainability.

- Clean and maintainable code
- Layered architecture
- Reusable repository/service methods
- Proper schema validation
- Readable Swagger/OpenAPI documentation
- Dockerized local execution
- Basic exception handling
- Simple but working prototype preferred over over-engineered design

Guidelines

- Follow best coding practices.
- Use proper folder structure and naming conventions.
- Keep the design extensible for future modules such as admin features.
- Add comments where useful for understanding.
- Make reasonable assumptions if some details are not explicitly mentioned.
- Focus on a working functional API over UI or advanced security.
- Use Git to maintain project repository, manage pull requests, and resolve conflicts.

Milestones

Milestone#	Milestone Description
Milestone 1	FastAPI project setup, folder structure, and database models
Milestone 2	Implement user registration/login and category/product browsing/search APIs
Milestone 3	Implement cart management APIs with validations
Milestone 4	Implement checkout, order placement, order history, and order details APIs

Milestone 5	Add basic exception handling wherever relevant
Milestone 6	Push the build to your GitHub repository.